COURSE SE515

NICOLAS BARBOUT

GRENOBLE INP UGA Esisar VALENCE

REAL-TIME NOISE LEVEL MEASUREMENT SYSTEM WITH HARDWARE-ACCELERATED LOGARITHM ON ZYBO Z7

MIAN MUHAMMAD NASIR

BUSHRA SHAFQAT

MASA RAFIE

CONTENT

INTRODUCTION

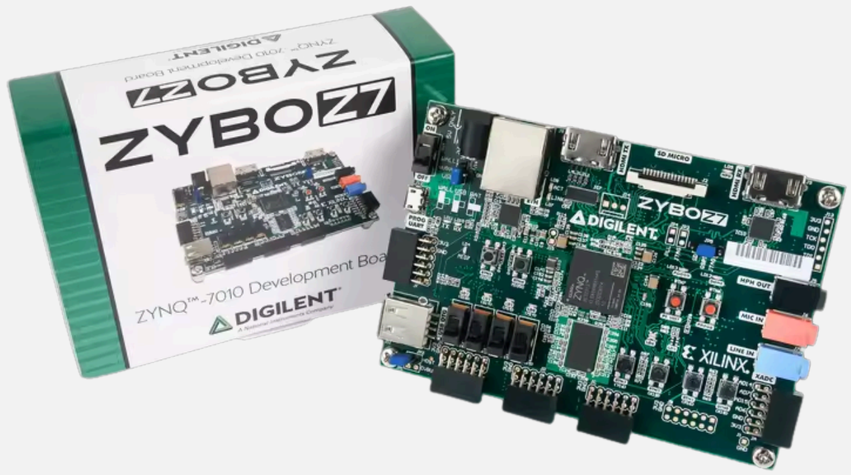


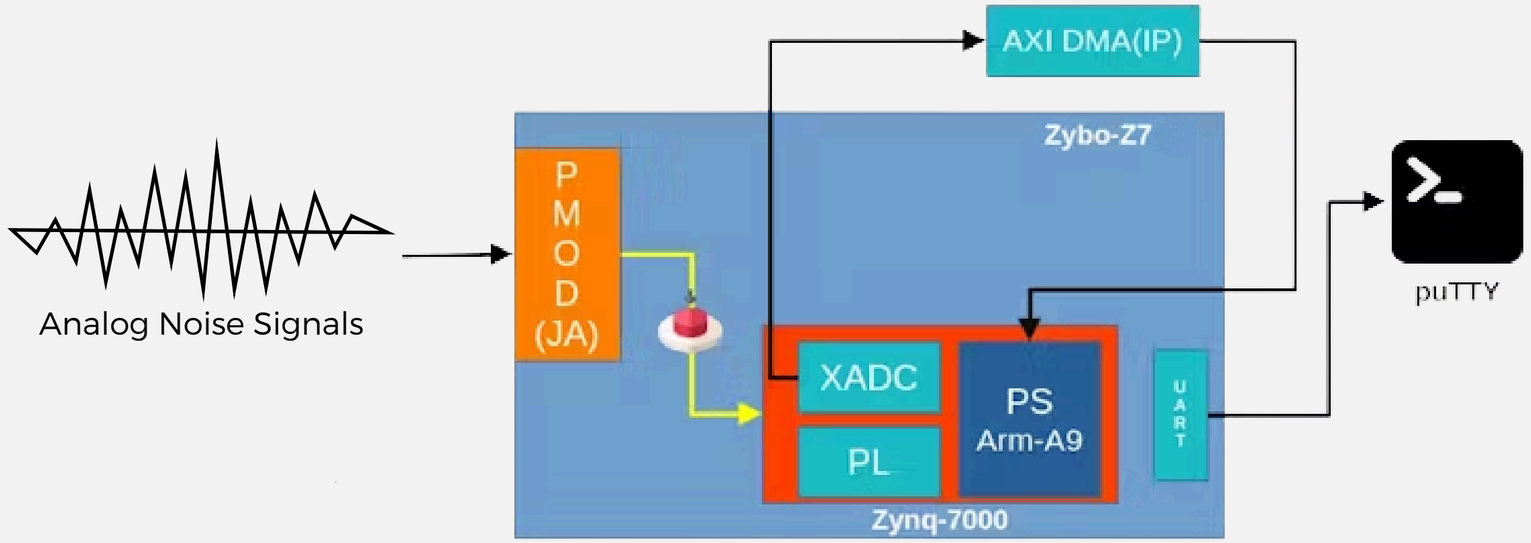
Figure 1: Zybo Z7-10

- Noise level monitoring is essential in environmental, industrial, and acoustic systems
- Sound intensity is expressed in decibels (dB), which requires logarithmic computation
- Real-time dB calculation is computationally expensive on embedded processors
- Efficient solutions are needed for real-time and low-power systems

OBJECTIVES

- Design a real-time noise level measurement system in dB
- Implement a NEON-optimized version
- Interface the onboard ADC to acquire analog noise signals.
- Design a hardware accelerator in the FPGA fabric to compute 10 × log10(value).
- Compute mean signal power on the ARM Cortex-A9 Processing System.
- Time Difference Comparison

BLOCK DIAGRAM



Hardware Acceltor / Neon

Figure 1: Block diagram

DESIGN SIMULATION

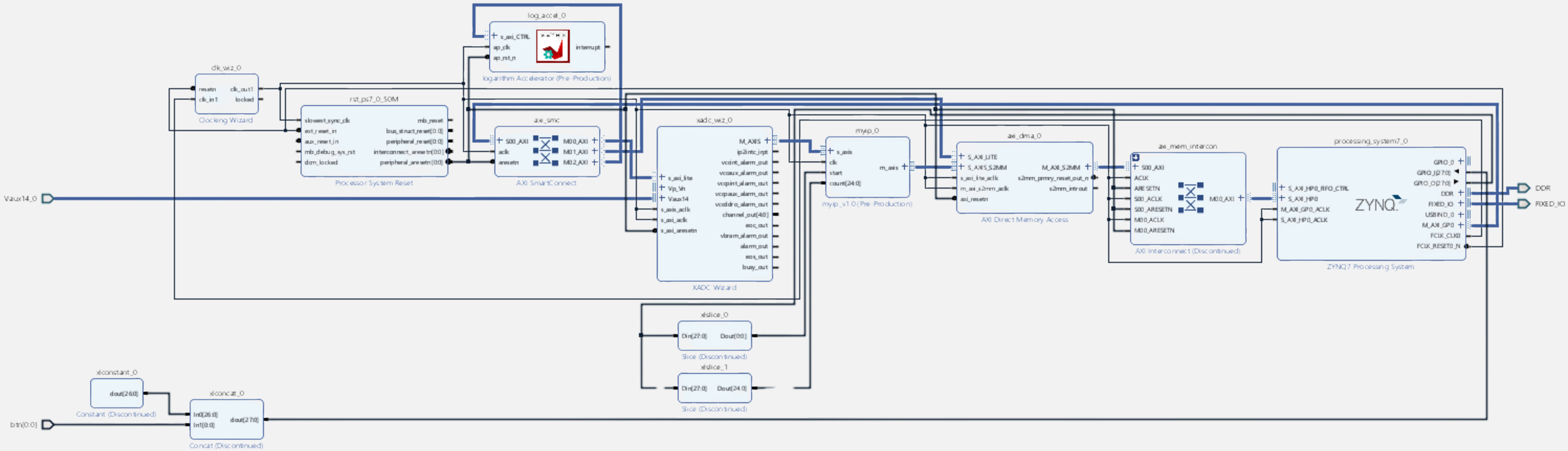
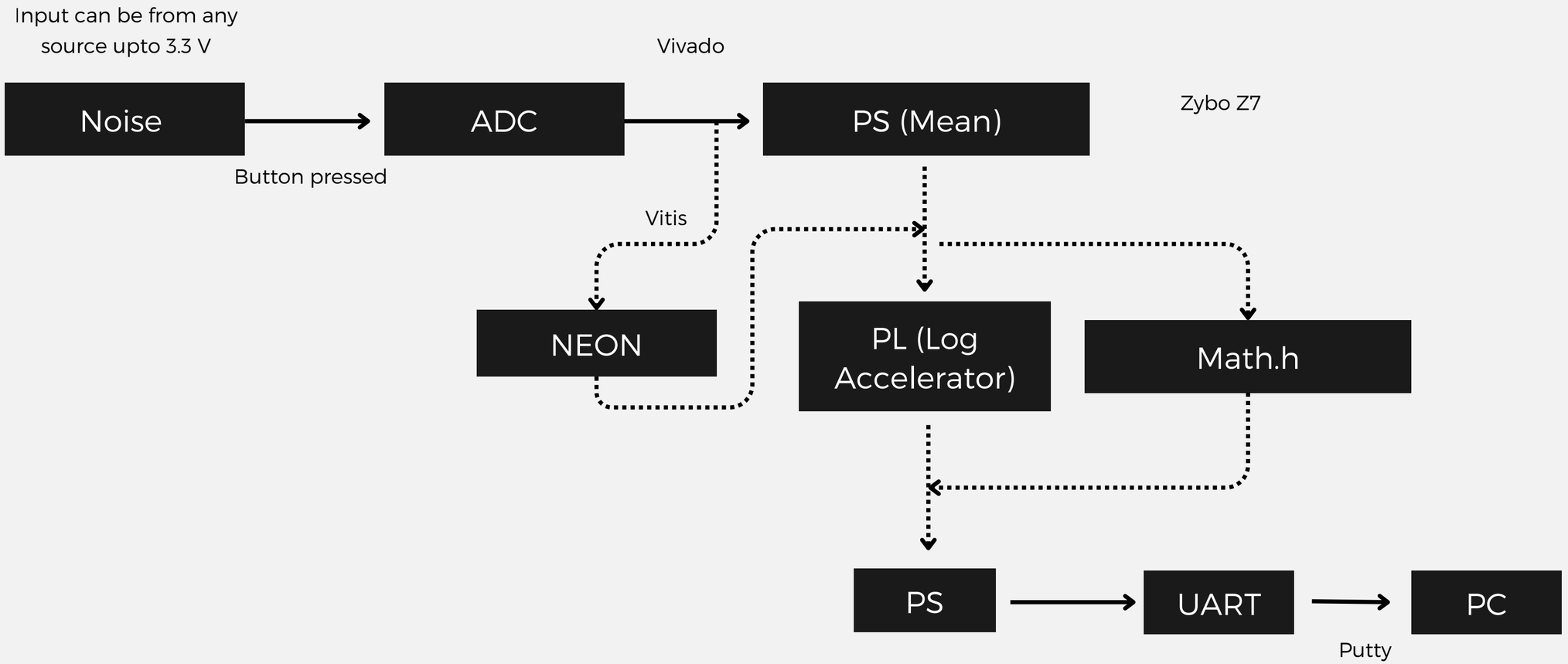


Figure 2: Design Simulation

METHODOLOGY

Noise signals are sampled using an ADC and processed on the ARM Cortex-A9 to compute mean power. The dB value is calculated using software, NEON optimization, and an FPGA-based hardware accelerator for performance comparison.



NEON

ARM NEON SIMD is used to accelerate software-based signal processing on the PS by computing the mean power of ADC samples in parallel. The logarithmic operation (10 × log10) is still performed in software using standard math functions, allowing performance comparison with the FPGA-based hardware accelerator.

```
190             }
191
192
193     // neon code
194
195         float result = square_and_sum_neon(voltage_array);
196         printf("Total sum = %f\n", result);
197
198                 float mean_square = sum_squares / SAMPLE_COUNT;
199                 float avg_power_db = log10_piecewise(mean_square);
200                 printf("Average power in dB = %.3f\n", avg_power_db);
201
202                 for (volatile int delay = 0; delay < 1000000; delay++);
203             }
204         }
205         return 0;
206     }
207
```

Figure 3: Neon computation

```
float square_and_sum_neon(const float *arr) {
    float32x4_t sum_vec = vdupq_n_f32(0.0f);
    int i = 0;

    for (; i <= SAMPLE_COUNT - 4; i += 4) {
        float32x4_t v = vld1q_f32(&arr[i]);
        float32x4_t squared = vmulq_f32(v, v);
        sum_vec = vaddq_f32(sum_vec, squared);
    }

    float temp[4];
    vst1q_f32(temp, sum_vec);
    float sum = temp[0] + temp[1] + temp[2] + temp[3];

    return sum;
    return 0 ;
}
```

Figure 4: Mean Calculation in Neon

HARDWARE ACCELERATOR

A custom FPGA hardware module is implemented in the Programmable Logic to compute 10 × log10 using fixed-point arithmetic. The PS sends the mean power via AXI, and the accelerator performs low-latency computation and returns the dB value, reducing CPU load and improving real-time performance.

helloworld.c 3 | Disassembly

src > helloworld.c > ...

```
// ================== HARDWARE LOG CALL ==================
/*
 * Start timing here:
 * Measures hardware accelerator invocation latency
 */
uint32_t start_cycles = get_cycle_count();

Xil_Out32(LOG_INPUT_ADDR, float_to_u32(mean_square)); // Send input
Xil_Out32(LOG_CTRL_ADDR, 0x01);                       // Start accelerator

// Wait until hardware sets DONE bit
while (!(Xil_In32(LOG_CTRL_ADDR) & 0x2));

uint32_t end_cycles = get_cycle_count();
// ======================================================

float avg_power_db = u32_to_float(Xil_In32(LOG_OUTPUT_ADDR));

/*
 * Compute and print hardware computation time
 */
uint32_t hw_cycles = end_cycles - start_cycles;
float hw_time_us =
    (float)hw_cycles / (SYSTEM_CLOCK_FREQUENCY_HZ / 1e6);
```

Figure 5: Hardware LOG

File Edit Selection View Go Terminal Vitis Help

VITIS EXPLORER

TESTING_HARDWARE_ACCELETOR - VITIS COMPON...

- log_accel_hls [HLS]
  - Settings
  - Includes
  - Sources
  - Test Bench
  - Output
- platform_IDE_LOG [Platform]

Welcome | tb_log_accel.cpp | vitis-comp.json | hls_config.cfg

log_accel_hls > tb_log_accel.cpp > ...

```
1  #include <cstdio>
2  #include <cmath>
3
4  extern "C" void log_accel(float x, float *y);
5
6  int main() {
7      float inputs[] = {1.0f, 10.0f, 100.0f, 0.5f, 0.001f};
8      for (int i=0;i<5;i++){
9          float x = inputs[i];
10         float y = 0.0f;
11         log_accel(x, &y);
12         printf("x=%f -> y=%f expected=%f\n", x, y, 10.0f*log10f(x));
13     }
14     return 0;
15 }
```

Figure 6: Hardware Accelerator

SOFTWARE

The Processing System acquires ADC samples and computes the mean power using a software-based implementation in C. The logarithmic calculation (10 × log10) is performed using standard math library functions, serving as a baseline for performance comparison with NEON optimization and hardware acceleration.

```c
float sum_squares = 0;
for (int i = 0; i < SAMPLE_COUNT; i++) {
    u16 raw = XSysMon_GetAdcData(&XADCInstance, XSM_CH_AUX_MIN + 14);
    float voltage = Xadc_RawToVoltageFunc(raw);
    voltage_array[i] = voltage;
    sum_squares += voltage * voltage;
    printf("VAUX14 Voltage: %.7f V\n", voltage);
}
// -------- START TIMING --------
    XTime_GetTime(&t_start);

float mean_square = sum_squares / SAMPLE_COUNT;

float avg_power_db;
if (mean_square <= 0.0f)
    avg_power_db = -100.0f;
else
    avg_power_db = 10.0f * log10f(mean_square);

printf("Average power in dB = %.3f\n", avg_power_db);

// -------- END TIMING --------
XTime_GetTime(&t_end);

time_us = (double)(t_end - t_start) * 1e6 / 1000000;
printf("Computation time for %d samples: %.3f us\n",
       SAMPLE_COUNT, time_us);

for (volatile int delay = 0; delay < 1000000; delay++);
```

Figure 7: Math.h library with mean computation

COMPARISONS

Software, NEON, and hardware-accelerated implementations are compared in terms of execution time and performance, with the hardware accelerator achieving the lowest latency.

Mean	Log	Time during processing
Software	Software	896 ms
Neon	Software	8.7 ms
Software	Hardware	0.656 ms
Neon	Hardware	

RESULTS

Method	Execution Speed	Hardware usage
Software(math.io)	Slow	None
NEON Optimized	Medium	None
PL Accelerator	Fast	FPGA logic

CONCLUSION

An efficient PS–PL co-design for real-time noise measurement is demonstrated, with hardware-accelerated logarithmic computation significantly improving performance.

Future Work:

- Higher sampling rates and dynamic range
- Improved log accuracy
- PC visualization
- Extended DSP acceleration

FAQ’S

Q1) Maximum Noise it can take as input?

Q2) What is the real-life usage of this project?

Q3) How you make the hardware accelerator? and why it was fast?